THE UNIVERSITY OF AZAD JAMMU AND KASHMIR

Department of Software Engineering

Student Name	**Fatima Arshad**
Roll Number	**2024-SE-24**
Lab Instructor	**Engr. Fahad Nisar**
Lab No	**01**
Subject	**Digital Image Processing**
Software Used	**Anaconda / Jupyter Notebook**

DIGITAL IMAGE PROCESSING LAB

LAB TASK # 01

1. Objective

The main objective of this lab is to learn the fundamentals of Python programming using Anaconda and Jupyter Notebook. This lab focuses on variables, data types, conditional statements, loops, functions, lists, tuples, dictionaries, and sets. It also aims to install and verify essential Python libraries, including NumPy, Pillow, and Matplotlib, and practise importing these libraries for digital image processing tasks.

2. Working Environment and Setup

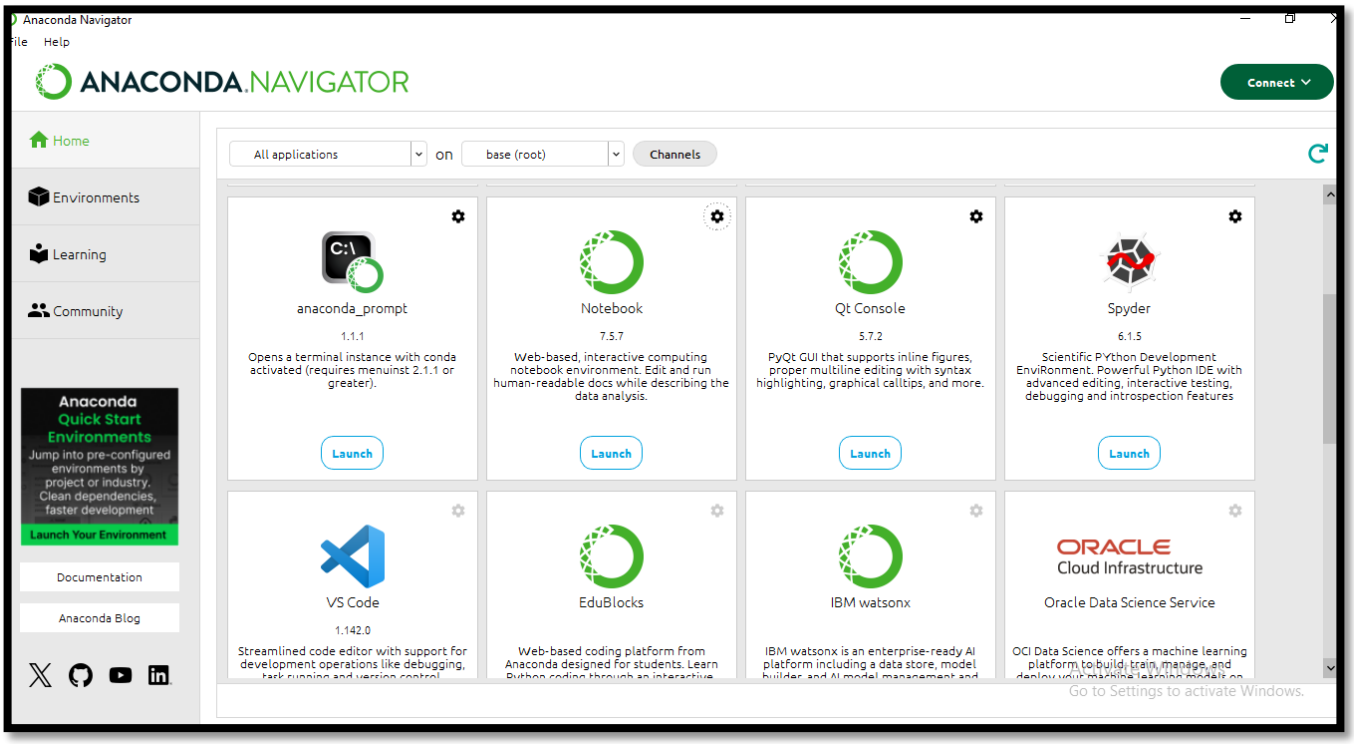
This lab was performed using Anaconda and Jupyter Notebook on a Windows computer. A project folder named dip-lab-01 was created inside the Documents directory to organize the lab files.

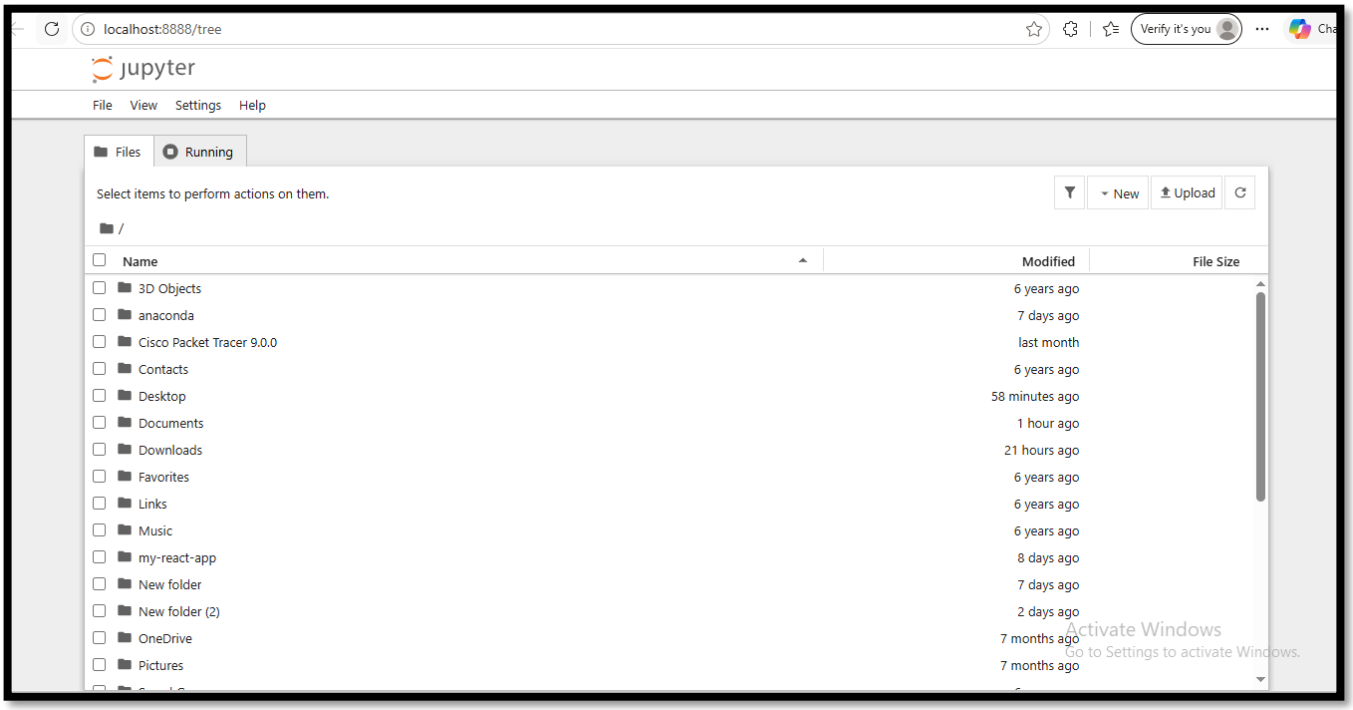
The Jupyter Notebook environment was used to write, execute, and test the Python programs. Three Python files were created:

- **hello.py** —> to display the first Python program output.
- **basics.py** —> to practise basic Python concepts through six exercises.
- **lab01_imports.py** —> to import the required Python libraries.

The Python version was 3.14.6, and the pip version was 26.1.2. The required libraries, NumPy, Pillow, and Matplotlib, were checked in the Anaconda environment. The package installation command confirmed that these libraries were already installed.

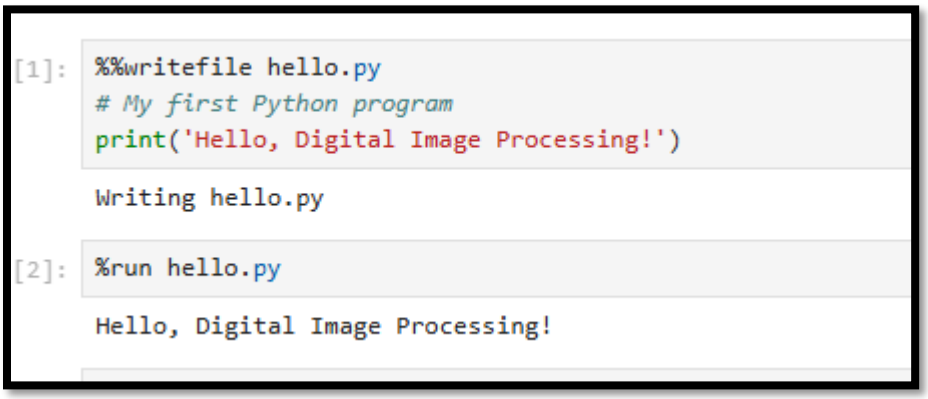
The programs were executed in Jupyter Notebook, and their outputs were checked to verify that the code worked as expected.





3. First Program — hello.py

The first program prints a confirmation message to verify that Python code can run successfully.



```
[1]: %%writefile hello.py
     # My first Python program
     print('Hello, Digital Image Processing!')

     Writing hello.py

[2]: %run hello.py

     Hello, Digital Image Processing!
```

4. Exercise 1 — Variables and Data Types

This exercise defines image width, height, name, and colour status. It prints each value and its data type, then calculates the total number of pixels.

- width = 640
- height = 480
- image_name = "sample.png"
- is_color = True

- print("Width:", width, type(width))
- print("Height:", height, type(height))
- print("Image Name:", image_name, type(image_name))
- print("Is Color:", is_color, type(is_color))
- print("Total Pixels:", width * height)

Expected key result: Total Pixels = **307200.**

```
23]: # Exercise 1: Variables and Data Types
     width = 640
     height = 480
     image_name = "sample.png"
     is_color = True

     print("Width:", width, type(width))
     print("Height:", height, type(height))
     print("Image Name:", image_name, type(image_name))
     print("Is Color:", is_color, type(is_color))
     print("Total Pixels:", width * height)
```

```
Width: 640 <class 'int'>
Height: 480 <class 'int'>
Image Name: sample.png <class 'str'>
Is Color: True <class 'bool'>
Total Pixels: 307200
```

```
[ ]:
```

5. Exercise 2 — if, elif, and else

Pixel intensity is valid when it is between 0 and 255 inclusive. Values below 128 are labelled Dark pixel; values from 128 through 255 are labelled Bright pixel.

- intensities = [-1, 0, 127, 128, 255, 256]

- for pixel in intensities:
- if pixel < 0 or pixel > 255:
- label = "Invalid"
- elif pixel >= 128:
- label = "Bright pixel"
- else:
- label = "Dark pixel"

- print(pixel, ":", label)

Expected labels: Invalid, Dark pixel, Dark pixel, Bright pixel, Bright pixel, Invalid.

```
[22]: # Exercise 2: if, elif and else
intensities = [-1, 0, 127, 128, 255, 256]

for pixel in intensities:
    if pixel < 0 or pixel > 255:
        label = "Invalid"
    elif pixel >= 128:
        label = "Bright pixel"
    else:
        label = "Dark pixel"

    print(pixel, ":", label)
```

```
-1 : Invalid
0 : Dark pixel
127 : Dark pixel
128 : Bright pixel
255 : Bright pixel
256 : Invalid
```

6. Exercise 3 — for and while Loops

- pixels = [20, 130, 255]

- for pixel in pixels:
- if pixel >= 128:
- print(pixel, ": Bright pixel")
- else:
- print(pixel, ": Dark pixel")

- counter = 0
- while counter < 3:
- print(counter)
- counter += 1

The for loop labels each pixel using 128 as the threshold. The while loop prints 0, 1, and 2. The counter is increased by one each time so the loop stops.

```
[24]: # Exercise 3: for and while loops
      pixels = [20, 130, 255]

      for pixel in pixels:
          if pixel >= 128:
              print(pixel, ": Bright pixel")
          else:
              print(pixel, ": Dark pixel")

      counter = 0
      while counter < 3:
          print(counter)
          counter += 1
```

```
20 : Dark pixel
130 : Bright pixel
255 : Bright pixel
0
1
2
```

7. Exercise 4 — Functions

- def is_bright(pixel, threshold=128):
- return pixel >= threshold

- print(is_bright(127))
- print(is_bright(128))
- print(is_bright(200, 210))

```
[25]: # Exercise 4: Functions
      def is_bright(pixel, threshold=128):
          return pixel >= threshold

      print(is_bright(127))
      print(is_bright(128))
      print(is_bright(200, 210))
```

```
False
True
False
```

The function returns True when the pixel intensity is at least the threshold; otherwise, it returns False.

8. Exercise 5 — Lists and Tuples

- pixels = [0, 128, 255]
- print("First item:", pixels[0])

- pixels[1] = 150
- pixels.append(64)

- print("Updated list:", pixels)
- print("Length:", len(pixels))

- size = (640, 480)
- width, height = size
- print("Width:", width)
- print("Height:", height)

A list is mutable, but a tuple is immutable.

A list is mutable, so its items can be changed after creation. A tuple is immutable, so its items cannot be reassigned after creation. Expected updated list: [0, 150, 255, 64]; length: 4.

```
26]: # Exercise 5: Lists and Tuples
pixels = [0, 128, 255]
print("First item:", pixels[0])

pixels[1] = 150
pixels.append(64)

print("Updated list:", pixels)
print("Length:", len(pixels))

size = (640, 480)
width, height = size
print("Width:", width)
print("Height:", height)

# A list is mutable, but a tuple is immutable.
```

```
First item: 0
Updated list: [0, 150, 255, 64]
Length: 4
Width: 640
Height: 480
```

9. Exercise 6 — Dictionaries and Sets

- image = {"width": 640, "height": 480}
- image["mode"] = "RGB"

- print("Width:", image["width"])
- print("Mode:", image["mode"])

- labels = ["cat", "dog", "cat", "bird"]
- unique_labels = set(labels)

- print("Unique labels:", unique_labels)
- print("Number of unique labels:", len(unique_labels))

A dictionary stores values using keys. A set keeps unique values, so the repeated label 'cat' is counted once. The expected number of unique labels is 3. The order in which a set is displayed may vary.

```
]: # Exercise 6: Dictionaries and Sets
image = {"width": 640, "height": 480}
image["mode"] = "RGB"

print("Width:", image["width"])
print("Mode:", image["mode"])

labels = ["cat", "dog", "cat", "bird"]
unique_labels = set(labels)

print("Unique labels:", unique_labels)
print("Number of unique labels:", len(unique_labels))

Width: 640
Mode: RGB
Unique labels: {'bird', 'dog', 'cat'}
Number of unique labels: 3

]:
```

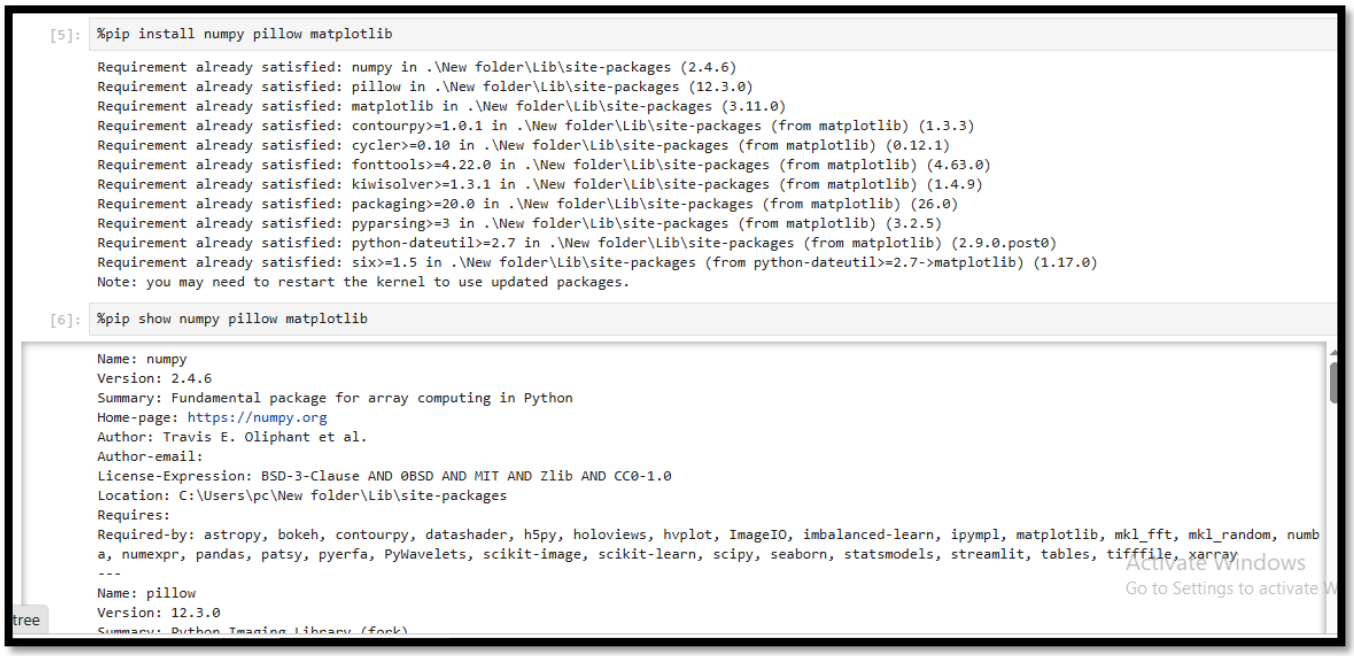
10. Required Packages

NumPy, Pillow, and Matplotlib were checked in the Anaconda/Jupyter environment. The installation command reported that the packages were already satisfied, meaning they were already installed in that environment.

- %pip install numpy pillow matplotlib
- %pip show numpy pillow matplotlib

Package purposes:

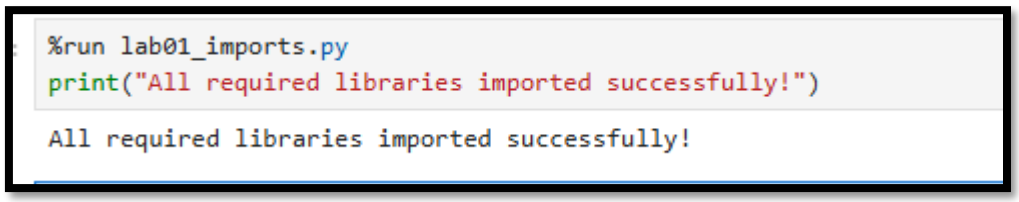
- NumPy: numerical operations and array handling.
- Pillow: image opening and image-processing tools.
- Matplotlib: plotting and visualising data.



11. Import Statement Verification

- import numpy as np
- from PIL import Image
- import matplotlib.pyplot as plt

The import statements make the required library features available to the program. The alias np is used for NumPy, Image is imported from PIL, and pyplot is imported as plt from Matplotlib. The import file ran without a Python exception. Matplotlib may display a first-run font-cache message; this is informational and is not itself an error.



12. Python vs. pip

Python is a programming language and runtime used to write and execute programs. Pip is a package installer that downloads and installs Python libraries. In this lab, pip was used to verify the availability of NumPy, Pillow, and Matplotlib.



```
[1]: import sys

     print("Python Version:", sys.version)

     Python Version: 3.14.6 | packaged by Anaconda, Inc. | (main, Jul  9 2026, 14:29:05) [MSC v.1942 64 bit (AMD64)]

[2]: %pip --version

     pip 26.1.2 from C:\Users\pc\New folder\Lib\site-packages\pip (python 3.14)
```

13. Conclusion

In conclusion, this lab provided practical experience with Python programming using Anaconda and Jupyter Notebook. The basic Python exercises helped develop an understanding of variables, data types, conditional statements, loops, functions, and data collections. The required libraries, including NumPy, Pillow, and Matplotlib, were also verified, and their import statements were tested successfully.

This lab established a foundation for future digital image processing tasks, including image loading, pixel manipulation, numerical operations, and image visualization.

THANK YOU!

****